

**Electrical and Computer Engineering Dept.
Oakland University
Automotive Mechatronics II
Winter 2026**

Term project

**Lane Keeping and Centering using
Raspberry Pi Scaled Robotic Car**

Name: George Trupiano

Submission Date: 04/22/2026

Instructor: Dr. Sebi

Term project	1
1. Abstract	3
2. Introduction	3
3. System Architecture Explanation	4
3.1. Layer Definitions	4
3.2. System Interface Diagram	4
4. Code Explanation	5
4.1. Application Logic:	5
4.1.1. Flow Diagram	5
4.2. Initialization Functionality:	6
4.2.1. Flow Diagram	6
4.3. ROI Calibration Startup Flow:	7
4.3.1. Flow Diagram	7
4.4. Camera Frame Capturing:	8
4.4.1. Flow Diagram	8
4.5. Ultrasonic Sensor Data Capturing:	9
4.5.1. Equation	9
4.5.2. Flow Diagram	9
4.6. Traffic Light Detection:	10
4.6.1. Flow Diagram	10
4.7. Traffic Light Distance Estimation:	11
4.7.1. Flow Diagram	11
4.8. Lane Detection:	12
4.8.1. Flow Diagram	12
4.9. Movement / Control Logic:	13
4.9.1. Object Detection	13
4.9.2. Traffic Light Response	14
4.9.3. Lane Keeping	15
4.9.4. Overall Flow Diagram	16
4.10. Shutdown Procedure:	17
4.10.1. Flow Diagram	17
5. Conclusion	19
6. References	20

1. Abstract

This project focuses on understanding, designing, and implementing a simplified Advanced Driver Assistance System (ADAS) using a Raspberry Pi 5, a camera, and an ultrasonic sensor. The purpose of this system is to simulate real-time vehicle assistance features such as lane detection, traffic light detection, object detection, and basic cruise control. Visual data is captured using a camera and processed to detect lanes and traffic lights, while distance measurements are obtained using an ultrasonic sensor to detect nearby objects. These inputs are used to determine the movement of the robotic car through predefined control logic. The system continuously monitors the environment and makes real-time decisions to adjust speed and direction. The results demonstrate the ability to replicate core ADAS functionality using low-cost hardware while maintaining real-time responsiveness and reliable sensor integration.

2. Introduction

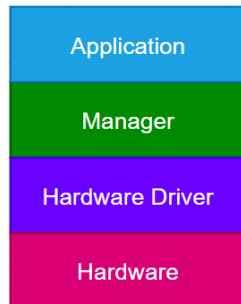
Advanced Driver Assistance Systems (ADAS) are automotive systems that assist drivers by monitoring the surrounding environment and making real-time decisions to improve safety. These systems commonly use a combination of sensors such as cameras and ultrasonic sensors along with onboard processing to detect objects, lanes, and traffic conditions. Based on this information, ADAS can provide warnings or automatically adjust vehicle behavior to prevent accidents.

The goal of this project is to implement a simplified version of an ADAS system using a Raspberry Pi 5–based robotic car. The system includes features such as lane detection, traffic light detection, object detection, and basic cruise control behavior. A camera is used to capture visual data for image processing tasks, while an ultrasonic sensor is used to measure distance to obstacles. The system processes this data in real time and applies control logic to determine motor behavior. By integrating these components, the project simulates how ADAS systems assist in vehicle navigation and safety without the use of more advanced control methods.

3. System Architecture Explanation

For the actual design of the software, the code is split into modules. Depending on the level that it interfaces with the system (directly controlling GPIO pins vs application level logic) dictates what layer they are present in.

3.1. Layer Definitions



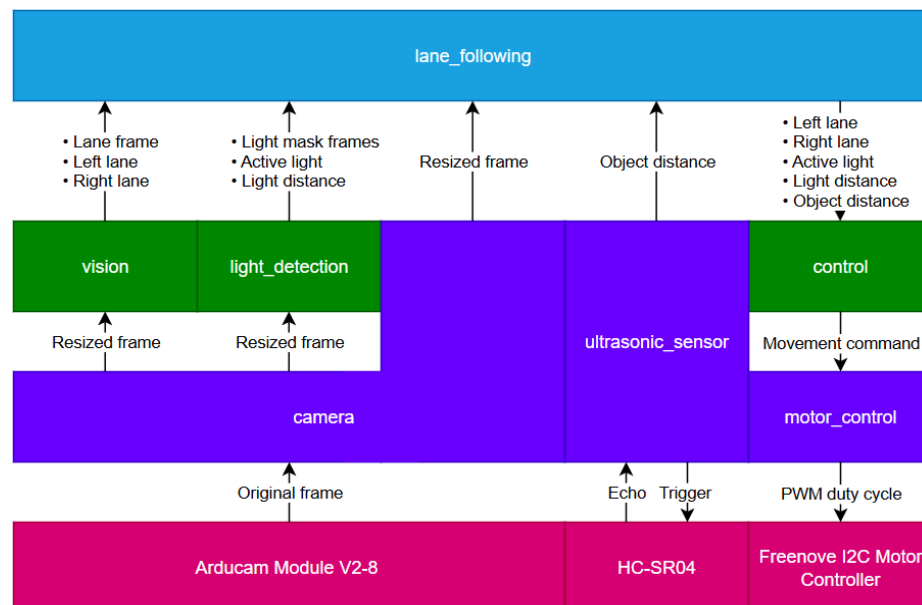
Application: Holds the main logic which interfaces with all other layers to execute the main features of the system

Manager: Contains feature level logic and handles interfacing with lower level logic to actually execute intended behavior.

Hardware Driver: Contains logic that directly interfaces with the hardware.

3.2. System Interface Diagram

This diagram describes how the different components interact with each other.



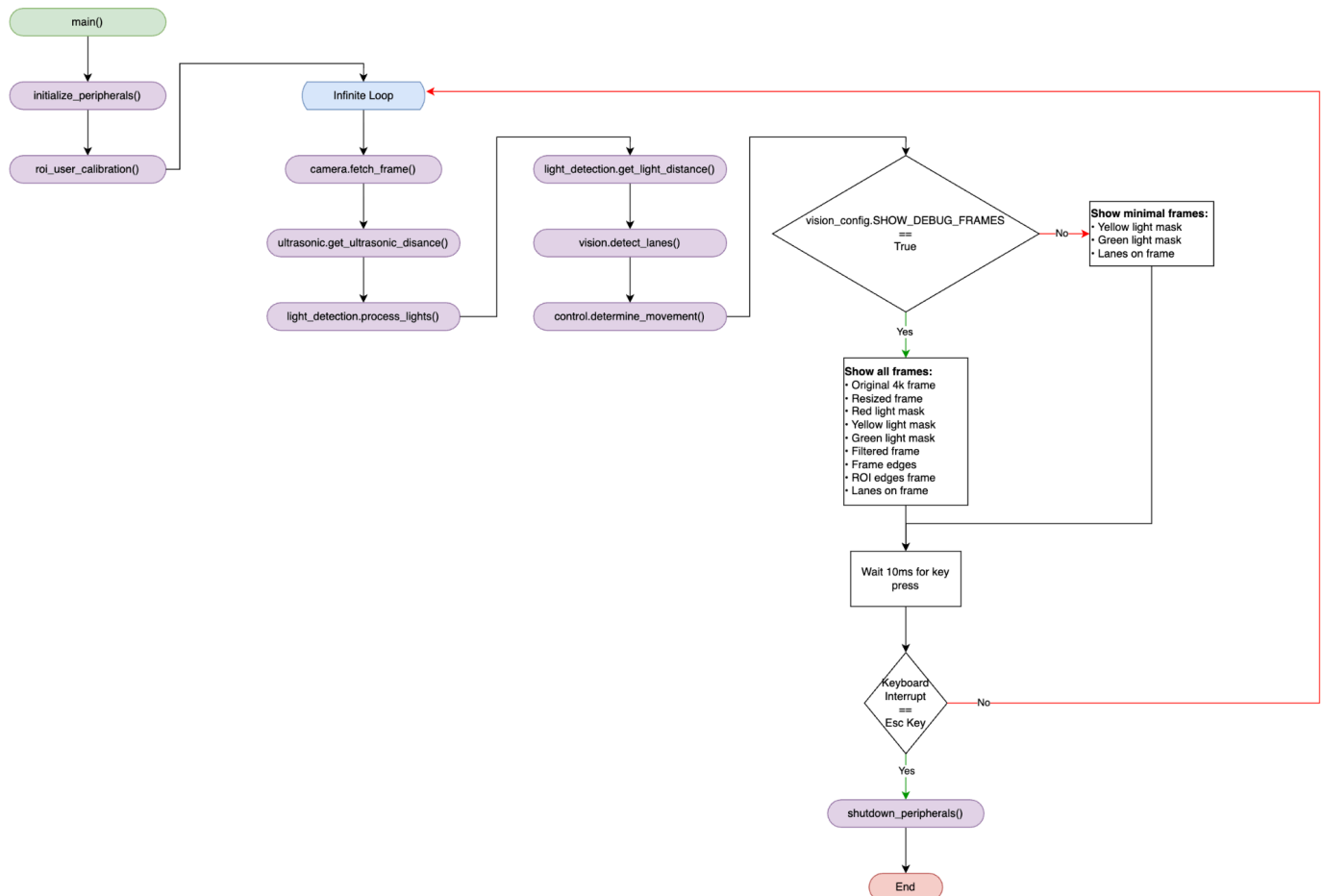
4. Code Explanation

This is a detailed overview of the program logic for all of the modules and features present within the system architecture.

4.1. Application Logic:

This function implements the overall application logic by using modules within the system in order to execute the required features. It configures all peripherals and allows users to calibrate the ROI. Then it loops over each frame and does the following sequence: capture peripheral data (camera, ultrasonic), execute image processing (detecting lanes, lights), commanding the robot to move properly based on the processed data, and showing the resulting data on the camera feed (lanes, lights).

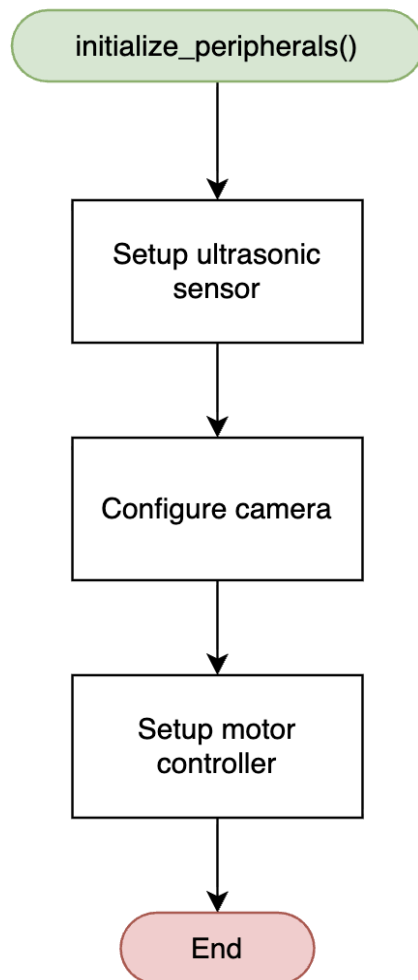
4.1.1. Flow Diagram



4.2. Initialization Functionality:

The initialization function configures all peripherals used in the system. These include the ultrasonic sensor, camera, and motor controller. Each peripheral is configured with the required settings and default parameters to operate within the system.

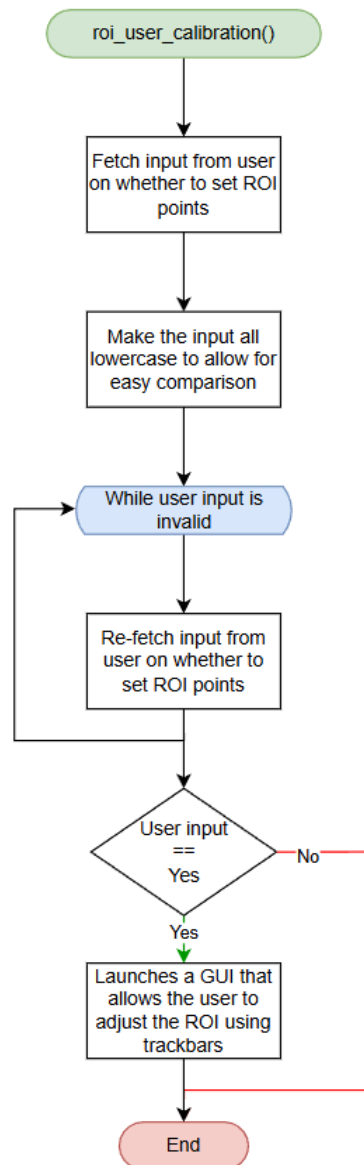
4.2.1. Flow Diagram



4.3. ROI Calibration Startup Flow:

The feature allows the user to dynamically change the ROI points before the main system starts up using a GUI. By displaying the camera feed and applying the ROI, it allows the user to see what ROI points would be optimal for the current robot setup. If the camera had changed height or angle for example, the ROI would need to be reconfigured which this would assist with.

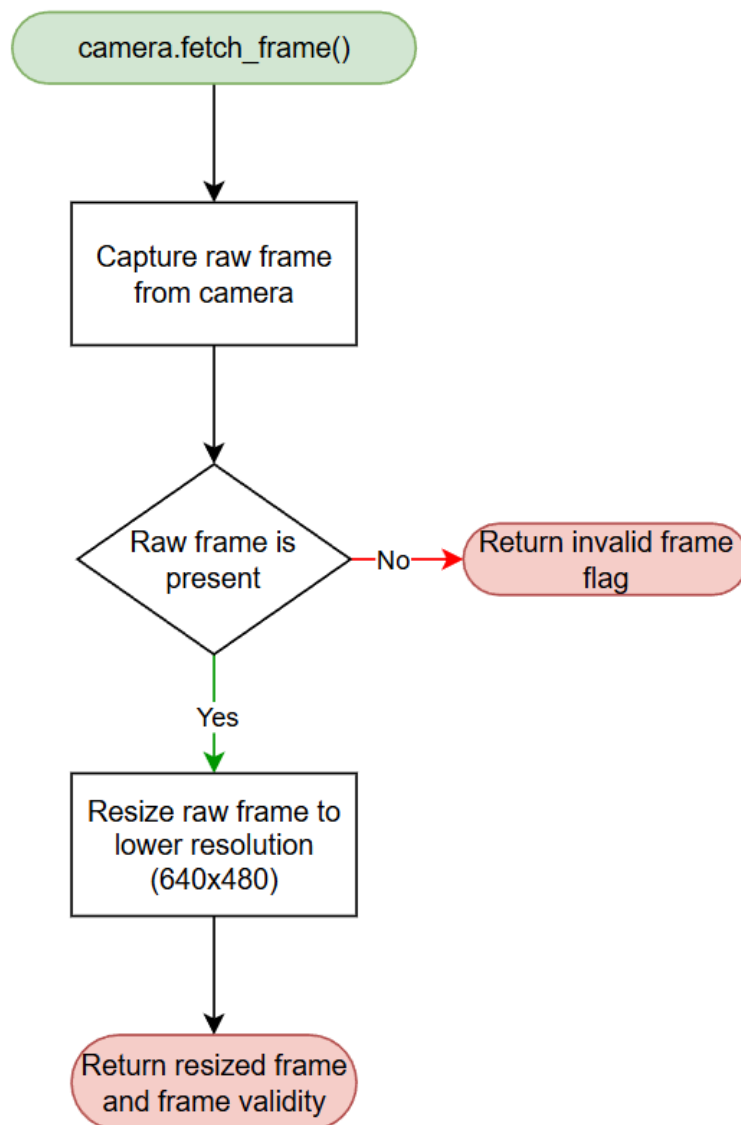
4.3.1. Flow Diagram



4.4. Camera Frame Capturing:

This module handles interfacing with the camera module. The module captures the raw frame from the camera and verifies that it exists before proceeding with any image processing. Originally, the camera defaults to outputting a 3840 x 2160 image. Due to the size, the speed at which the Raspberry Pi can process each frame is impacted significantly. As a result of this, once the raw frame is captured, that frame is then resized to a lower resolution of 640x480 to keep up a fast processing speed.

4.4.1. Flow Diagram



4.5. Ultrasonic Sensor Data Capturing:

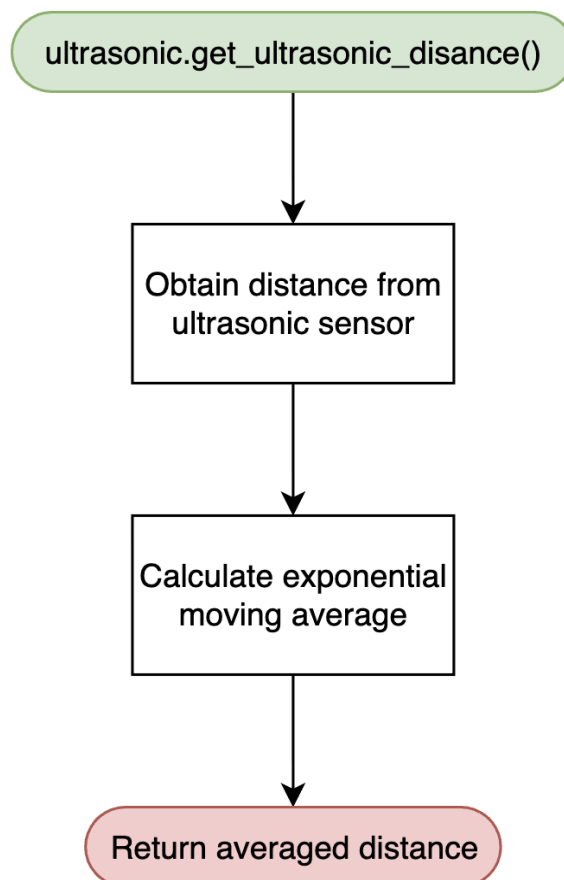
This module handles interfacing with the ultrasonic sensor. The module obtains distance from the ultrasonic sensor and applies an exponential moving average to the raw distance reading from the sensor. By doing this, it allows the sensor reading to be smoothed for more decisive behavior.

4.5.1. Equation

$$z_{\text{New Running Average}} = (\alpha * x_{\text{instantaneous}}) + ((1 - \alpha) * y_{\text{previous average}})$$

What this calculation allows is to control the amount of impact the new instantaneous value has on the running average.

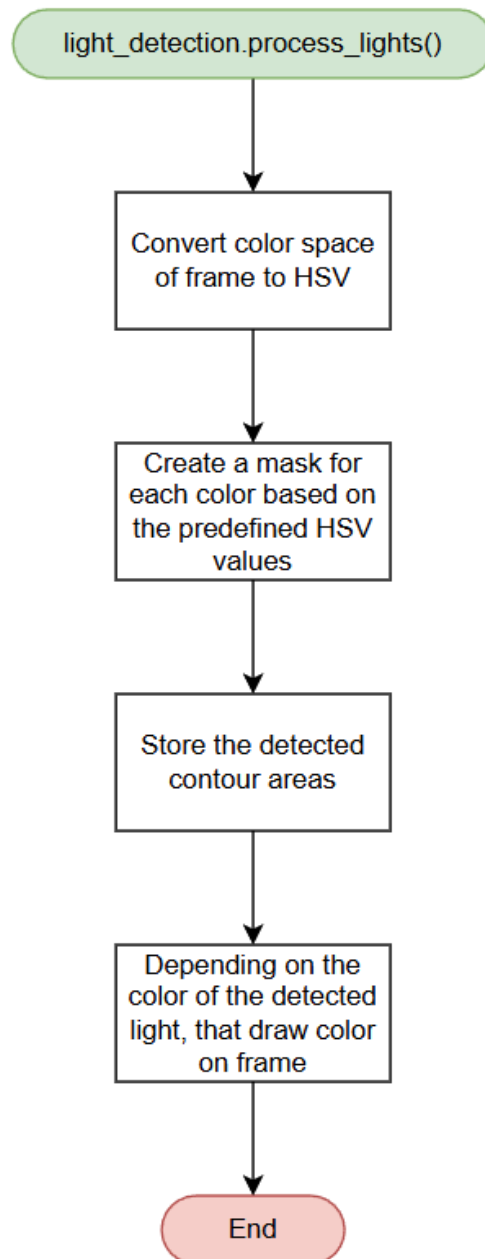
4.5.2. Flow Diagram



4.6. Traffic Light Detection:

The light detection module uses predefined HSV ranges for each color (red, yellow, and green) and applies those masks to the frame. Based on which mask yields results with the proper area constraints, a bounding box is drawn on the detected light and text with the detected light is displayed. It will also return the area for use later for distance calculation

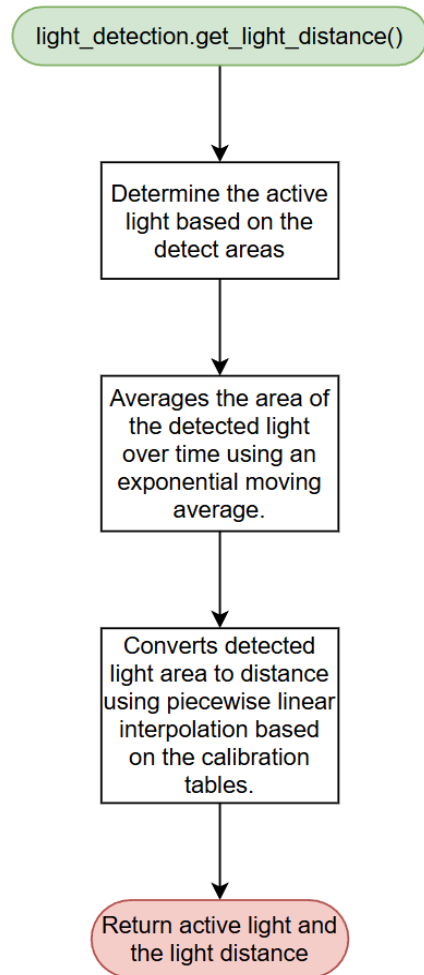
4.6.1. Flow Diagram



4.7. Traffic Light Distance Estimation:

The same light detection module handles determining which light is actively being detected as well as calculating the distance of the detected light. Determines the active light based on which color is detected. Then using the same approach for averaging the lanes, an exponential moving average is used on the detected light area. For determining the distance the light is from the car, a calibration table is used. Since the area doesn't scale with distance linearly, a different approach needed to be used. By mapping the area of each colored light to a real world distance for multiple points, piecewise interpolation can be used. This makes it so the math is always simpler linear interpolation even on a set of data that isn't linear. Points needed to be created at multiple distances for each color to get accurate results.

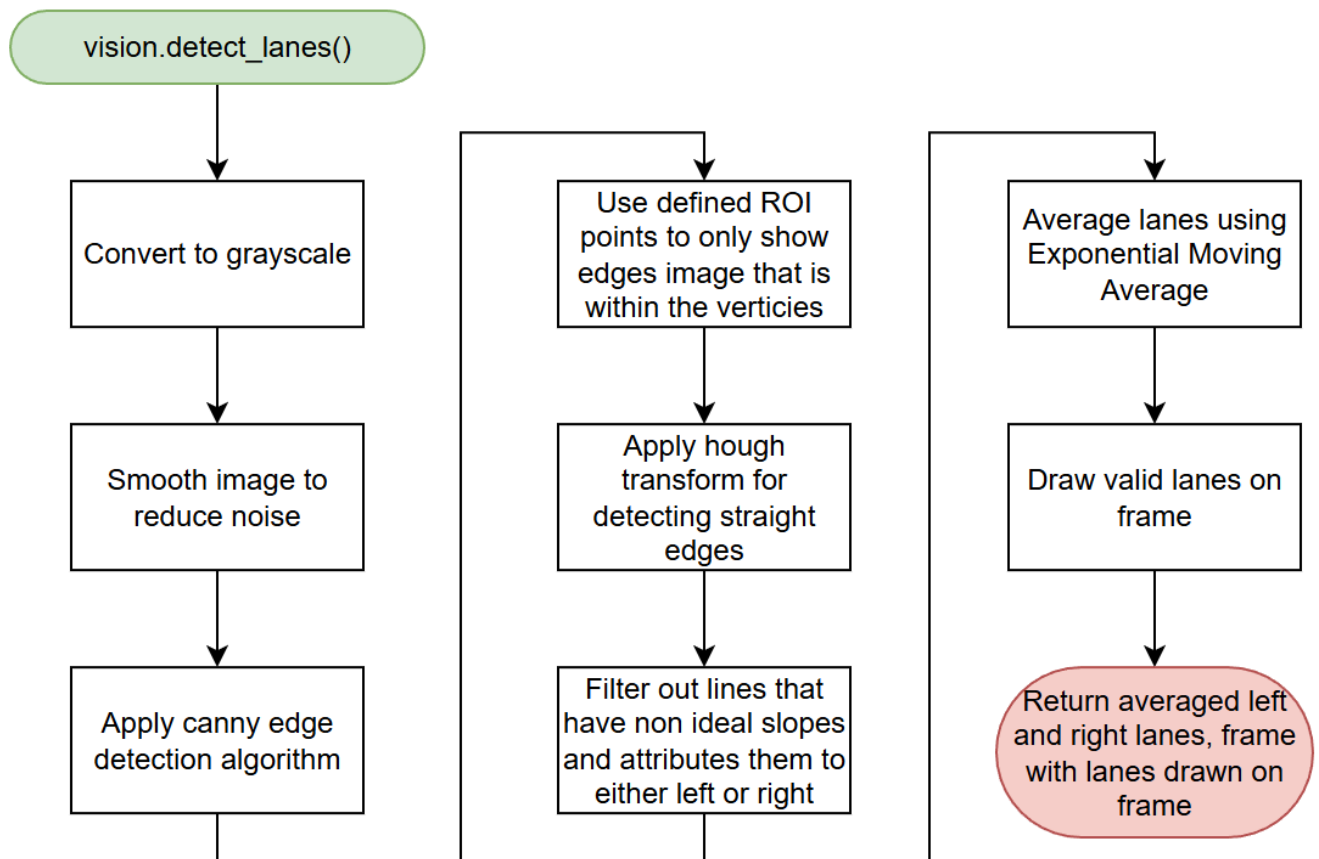
4.7.1. Flow Diagram



4.8. Lane Detection:

This module is in charge of detecting both the left and right lanes present within the frame. With those lanes detected, they are then drawn onto the original frame. The way it actually detects the lanes is with the algorithm described in the flow chart below.

4.8.1. Flow Diagram



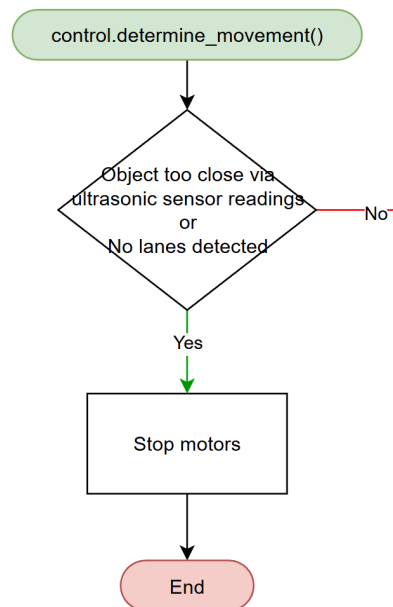
4.9. Movement / Control Logic:

This module is in charge of determining how the car should move based on the information collected during the image processing stages including (left and right lane coordinates, active light, light distance, and object distance). The control logic follows a priority based system which is:

4.9.1. Object Detection

The ultrasonic sensor is used to determine whether there is an obstacle blocking the path of the robot.

4.9.1.1. Object Detection Flow Diagram

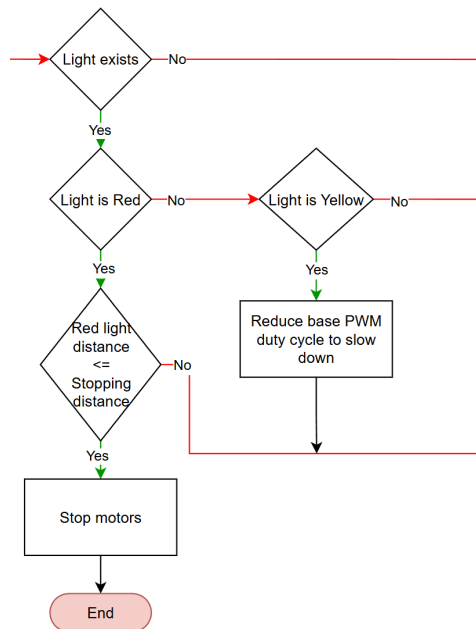


4.9.2. Traffic Light Response

The system responds differently based on which light it is currently detecting.

- When a red light is detected, the system checks to see if the robot is too close. If it is then the motors are commanded to stop.
- When a yellow light is detected, the system modifies the base PWM values for speed control to be significantly slower. The system will then proceed to the lane keeping logic with the modified PWM values.
- When a green light is detected, the system will proceed as normal to the lane keeping logic

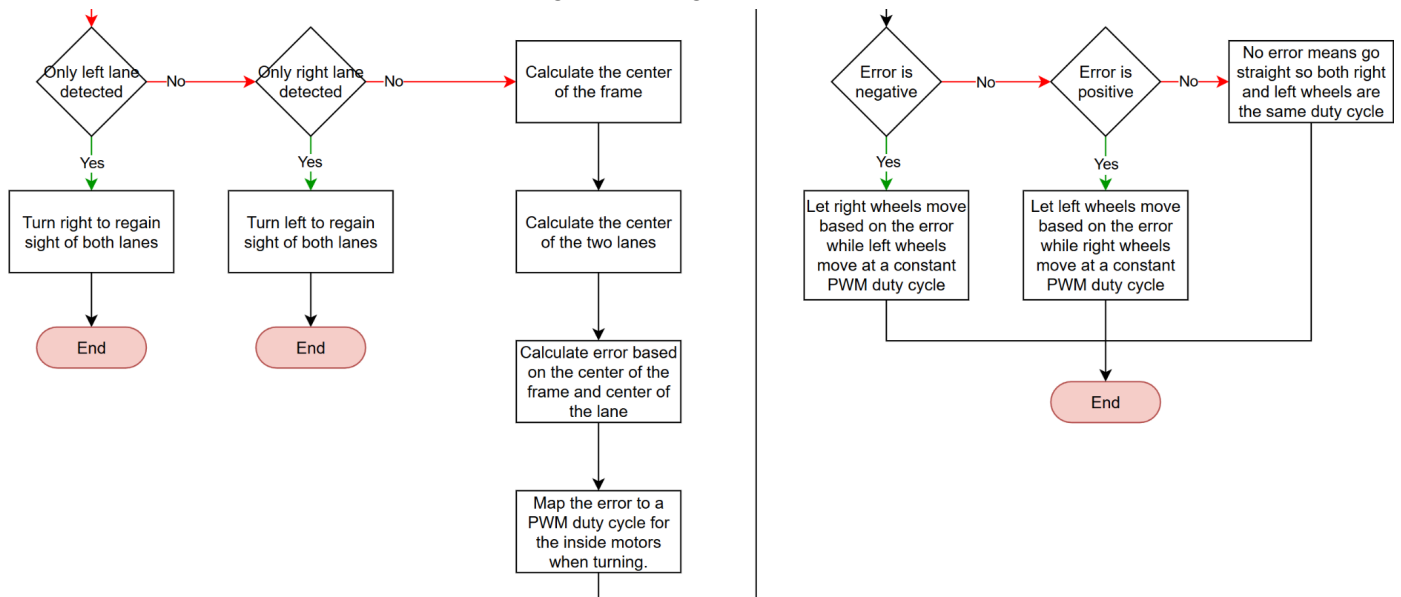
4.9.2.1. Traffic Light Flow Diagram



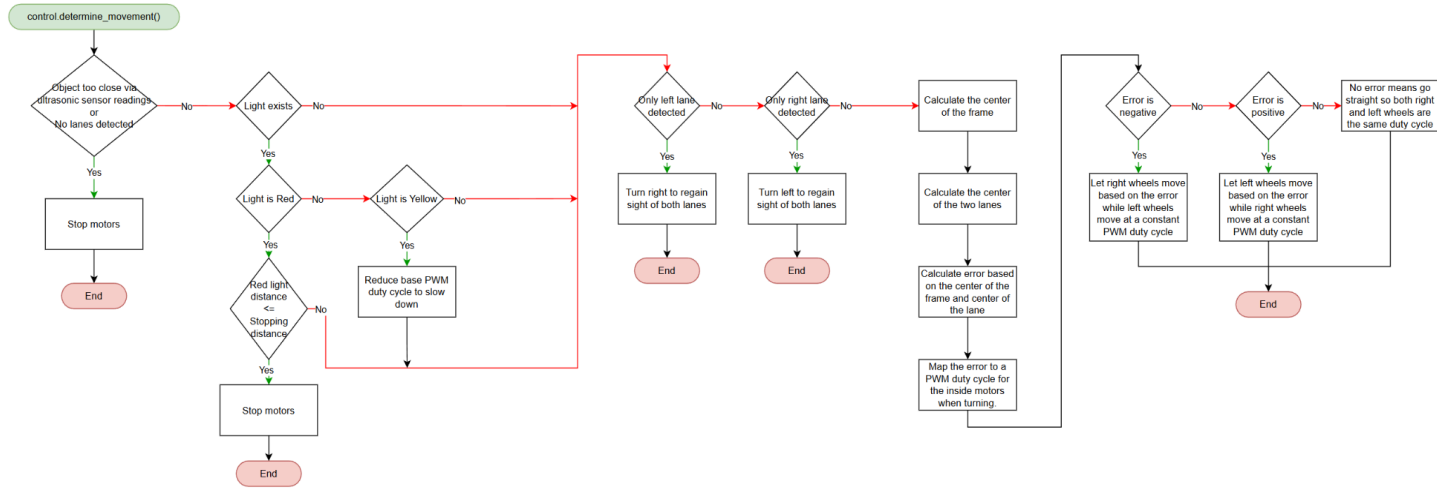
4.9.3. Lane Keeping

The system will attempt to keep the robot within the two detected lanes. There is an edge case that is important to address before determining how to keep the robot within the two lanes. An important edge case occurs when only one of the two lanes is detected. This was solved by having the robot do an arc turn at a set PWM duty cycle to the left or the right based on which lane was missing. By doing this, it should allow the robot to see both lanes. Now the system is allowed to proceed with the lane keeping logic. The approach is calculating the difference between the frame center and the lane center. Based on that error, the PWM duty cycle of the left or the right wheel is mapped to the error. Whether the error is mapped to the left or the right wheels is based on the sign of the error. With a negative error, the left wheels PWM duty cycle will be mapped to the error. Likewise, a positive error will map the error to the PWM duty cycle of the right wheels.

4.9.3.1. Lane Keeping Flow Diagram



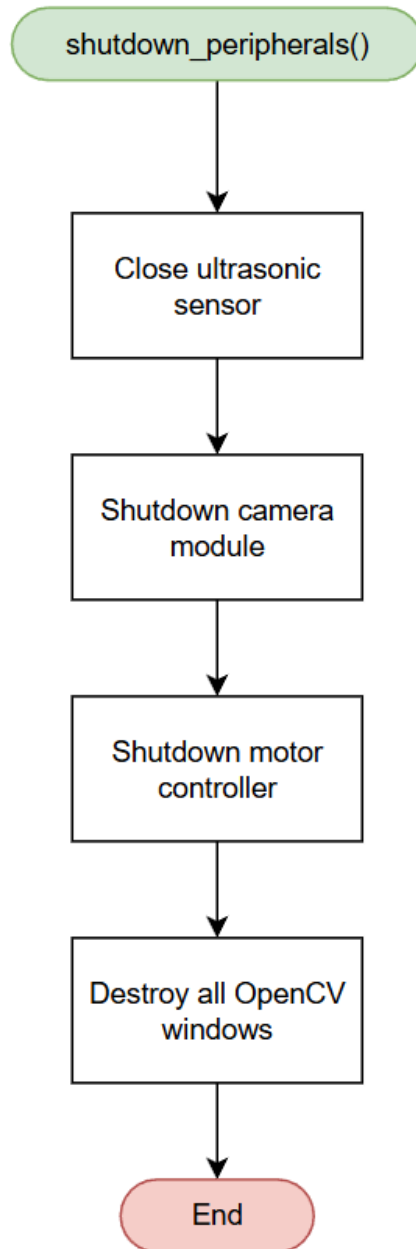
4.9.4. Overall Flow Diagram



4.10. Shutdown Procedure:

The shutdown function properly releases control of all peripherals so they will be set to a safe state before the program closes. This is done for the ultrasonic sensor, camera, motor controller, and OpenCV windows.

4.10.1. Flow Diagram



5. Conclusion

This project successfully implements a simplified Advanced Driver Assistance System (ADAS) using a Raspberry Pi 5, a camera, and an ultrasonic sensor. By using a modular architecture and a structured application flow, the system is able to capture sensor data, process visual information, and apply control logic to determine the movement of the robotic car. The overall approach follows the sequence shown in the application logic and flow diagrams, where the system initializes peripherals, allows ROI calibration, continuously captures camera and ultrasonic data, processes that data for lane and light detection, and then commands the motors accordingly .

The system uses techniques such as exponential moving average (EMA) filtering for both distance measurements and visual data smoothing, HSV-based color detection for traffic lights, and interpolation for estimating distance based on detected light area. The control logic is implemented using a priority-based approach where object detection, traffic light response, and lane keeping are handled in sequence. This allows the system to respond appropriately to different conditions such as stopping at red lights, slowing for yellow lights, and maintaining position within detected lanes.

While the system performs effectively in a controlled environment, there are limitations when considering real-world applications. These include sensitivity to lighting conditions, reliance on fixed HSV thresholds, and assumptions made in lane detection and distance estimation. Additionally, the lack of more advanced control methods such as PID or sensor fusion limits robustness in dynamic environments. Despite these limitations, the project demonstrates key concepts such as modular system design, real-time processing, sensor integration, and decision-based control logic, which are all fundamental to modern ADAS implementations.

6. References

- Raspberry Pi 5 Documentation
 - <https://www.raspberrypi.com/documentation/>
- Library Documentation for gpiozero
 - <https://gpiozero.readthedocs.io/en/stable/>
- OpenCV Documentation
 - <https://docs.opencv.org/4.x/>
- FreeNove FNK00043 Documentation
 - <https://docs.freenove.com/projects/fnk0043/en/latest/index.html>